

Univerzita Jana Evangelisty Purkyně
v Ústí nad Labem
Přírodovědecká fakulta



Porovnání databázových systémů pro
zpracování časových řad

BAKALÁŘSKÁ PRÁCE

Vypracoval: Martin Formánek

Vedoucí práce: Ing. Roman Vaibar, Ph.D., MBA

Studijní program: Aplikovaná informatika

Studijní obor: Aplikovaná informatika

ÚSTÍ NAD LABEM 2026

Namísto žlutých stránek vložte digitálně podepsané zadání kvalifikační práce poskytnuté vedoucím katedry.

Zadání musí zaujímat právě dvě strany.

Zadání je nutno vložit jako PDF pomocí některého nástroje, který umožňuje editaci dokumentů (se zachováním elektronického podpisu).

V Linuxu lze například použít příkaz `pdftk`.

Prohlášení

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně a použil jen pramenů, které cituji a uvádím v přiloženém seznamu literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., ve znění zákona č. 81/2005 Sb., autorský zákon, zejména se skutečností, že Univerzita Jana Evangelisty Purkyně v Ústí nad Labem má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Jana Evangelisty Purkyně v Ústí nad Labem oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladu, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

V Ústí nad Labem dne 29. dubna 2026

Podpis:

Děkuji vedoucímu práce Ing. Mgr. Romanovi Vaibarovi, Ph.D., MBA
za neocenitelné rady a pomoc při tvorbě bakalářské práce.

Dále děkuji své rodině a nejbližším přátelům
za podporu během studia a psaní závěrečné práce.

Abstrakt:

Abstract práce uvádí základní téma práce a především její výstupy. Rozsah abstraktu by měl být alespoň osmdesát slov (abstrakty a klíčová slova se však musí vejít na stránku).

Klíčová slova: tři až pět klíčových slov resp. termínů, která usnadní případné vyhledávání závěrečné práce, v pořadí od obecnějších ke konkrétnějším

COMPARISON OF DATABASE SYSTEMS FOR TIME SERIES PROCESSING

Abstract:

The abstract states the main topic of the thesis and mainly its outcomes. The length of the abstract should be at least eighty words (however, abstracts and keywords must fit on a page)

Keywords: three to five key words or terms to facilitate a possible search of the thesis, in order from more general to more specific

Obsah

Seznam obrázků	13
Seznam ukázek kódu	15
Seznam tabulek	17
1. Úvod	19
2. Teoretická část	21
2.1. Charakteristika časových řad	21
2.2. Ukládání časových řad	22
2.3. Databázové systémy	24
2.4. Úvod do problematiky testování a implementace	28
2.5. Ansible	30
2.6. Monitorování	37
2.7. Testování	42
3. Praktická část	47
3.1. Popis dat	47
3.2. Předběžná srovnávací analýza systémů	47
3.3. Návrh testovacího scénáře	49
3.4. Konfigurace testovacího prostředí a databázových systémů	50
3.5. Pilotní nasazení systémů	52
3.6. Implementace testů	53
4. Výsledky výzkumu a diskuse	55
5. Závěr	57
Seznam použitých zdrojů	60
A. Externí přílohy	61

Seznam obrázků

2.1.	Ukázka časové řady	22
2.2.	Ukázka struktury SQL databáze	25
2.3.	Ukázka struktury databáze párů klíč-hodnota	26
2.4.	Ukázka struktury grafové databáze	28
2.5.	Ukázka metriky typu Counter	39
2.6.	Ukázka metriky typu Gauge	39
2.7.	Ukázka metriky typu Histogram	40
2.8.	Ukázka dashboardu v Grafaně [15]	41
2.9.	Ukázka organizace dat v B-Tree struktuře [16]	44
2.10.	Ukázka organizace dat v LSM stromu [18]	45

Seznam ukázek kódu

2.1.	Redis - ukázka syntaxe	25
2.2.	MongoDB - ukázka syntaxe	26
2.3.	ClickHouse - ukázka syntaxe	27
2.4.	Neo4j - ukázka syntaxe	28
2.5.	Ansible - ukázka úlohy	31
2.6.	Ansible - ukázka playbooku	31
2.7.	Ansible - ukázka INI inventáře	32
2.8.	Ansible - ukázka YAML inventáře	33
2.9.	Struktura adresářů pro Ansible role	33
2.10.	Použití proměnné	35
2.11.	Shromážděná fakta	35
2.12.	Proměnné přidání argumentem	35
2.13.	Proměnné při zahrnutí role	36
2.14.	Registrované proměnné	36
2.15.	Playbook proměnné	36
2.16.	Proměnné hostů	36
2.17.	Proměnné skupin	36
2.18.	Proměnné v inventáři	37
2.19.	Prometheus - ukázka konfigurace	38
2.20.	PromQL - ukázka dotazů	40
3.1.	Struktura adresářů pro Ansible	50
3.2.	Příklad spuštění live insert testu	53
3.3.	Příklad spuštění live insert testu	54

Seznam tabulek

2.1.	Časová řada načítání webové stránky	24
2.2.	Zjednodušený záznam časové řady načítání webové stránky	24
2.3.	Grafana - ukázka data framu teploty CPU	42

1. Úvod

2. Teoretická část

2.1. Charakteristika časových řad

Časové řady představují sekvence datových bodů, které jsou zaznamenávány buď v pravidelných intervalech v čase, nebo kontinuálně. V případě, že jsou hodnoty měřeny v konkrétních a oddělených časových okamžicích, hovoříme o diskrétní časové řadě. Naopak, pokud lze veličinu považovat za definovanou v každém okamžiku času, jedná se o kontinuální časovou řadu.

Tyto datové body mohou představovat různé jevy, například teplotní záznamy, údaje z finančních trhů, senzorová měření, dopravní intenzitu, spotřebu energie nebo jiné sledované hodnoty. Nemusejí však být nutně číselné, mohou být také kategorické, například stav zařízení nebo typ události. Datové body také nemusejí být striktně naměřené v přesný okamžik, ale mohou představovat agregované hodnoty za určitý časový interval, například denní průměrnou teplotu nebo hodinový výstup pracovní linky. Délka těchto intervalů se volí podle povahy sledovaného jevu a požadavků analýzy a může sahát od sekund až po měsíce či roky. [1]

Analýza

V podstatě všechny časové řady jsou zaznamenávány za účelem následné analýzy. Tato analýza může mít různé cíle, které lze shrnout do následujících kategorií:

1. Zkoumání vlivů na sledovaný jev

Časové řady se často používají k odhalování vztahů mezi různými faktory a sledovaným jevem. Například v ekonomii mohou být analyzovány dopady změn úrokových sazeb na inflaci nebo v energetice vliv teploty na spotřebu elektrické energie. Taková analýza umožňuje pochopit, jak jednotlivé vlivy ovlivňují chování systému v čase.

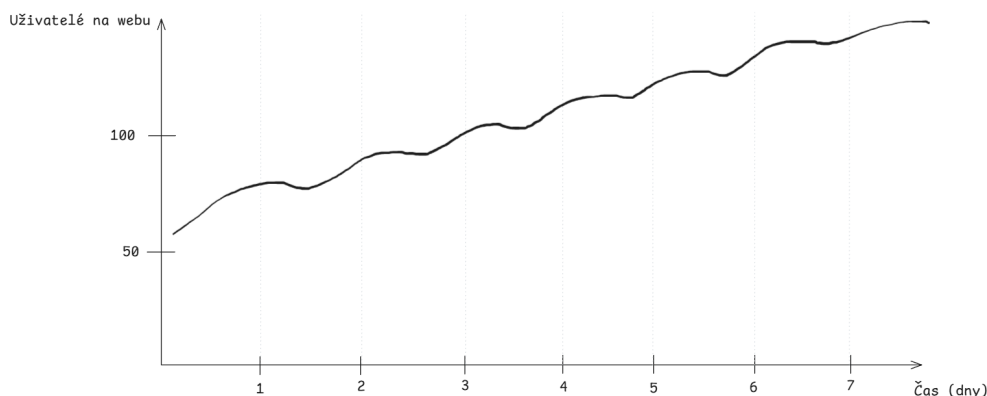
2. Studium samotného jevu

Někdy je cílem zkoumat pouze samotný jev. Například meteorologické časové řady teplot mohou odhalit dlouhodobé oteplování, sezónní výkyvy nebo extrémní jevy, jako jsou vlny horka.

3. Predikce budoucího vývoje

Historická data umožňují předpovídat budoucí hodnoty sledovaného jevu. Taková predikce se využívá například při plánování kapacit výroby, řízení zásob nebo finančních investic.

Analýza časových řad zahrnuje různé techniky a metody, které umožňují identifikovat vzory, trendy, sezónní výkyvy a další charakteristiky v datech. [1]



Obrázek 2.1.: Ukázka časové řady

Na obrázku 2.1 je znázorněna ukázka časové řady, kde na vodorovné ose jsou zobrazeny časové okamžiky v rámci dní a na svislé ose hodnoty sledovaného jevu - Současných online uživatelů na webovém portálu.

Lze na ní pozorovat denní cyklus, neboli sezónnost, kdy počet uživatelů vrcholí v odpoledních hodinách a klesá během noci, a zároveň linární rostoucí trend, stránku každý den navštíví více uživatelů.

2.2. Ukládání časových řad

Mnohdy hodnota časových řad leží spíše v retrospektivní analýze než v analyzování každé nové hodnoty v reálném čase. Z tohoto důvodu je často potřeba tyto časové řady efektivně ukládat pro pozdější analýzu. Existuje mnoho různých přístupů k ukládání časových řad, od tradičních relačních databází přes specializované databázové systémy navržené speciálně pro tento účel až po obyčejné textové soubory.

Při výběru správného formátu a databázového systému pro ukládání časových řad je důležité zvážit několik faktorů, jako jsou:

- **Objem dat**

Časové řady mohou a mnohdy i generují daleko větší objemy dat než jiné typy dat. Tento objem je důležité předem odhadnout a zvolit systém, který je schopný efektivně tento růst zvládnout.

- **Neustálý příjem nebo jednotlivé události**

Při vybírání ideálního systému, například pro ukládání návštěvnosti webových stránek, je potřeba zvážit charakter dat. V případě neustálého toku dat je obvykle nejzajímavější pracovat s nejnovějšími záznamy, protože ty starší mohou být méně relevantní pro analýzu.

Naopak pokud data představují jednotlivé události, například každoroční dopravní provoz během svátků, může být důležité mít přístup i k historickým datům. V takovém případě je pravděpodobnější, že systém bude potřebovat podporu pro náhodný přístup k různým časovým intervalům.

- **Trvání projektu**

Pokud je projekt krátkodobý, může být dostačující použít jednodušší a rychle nasaditelná řešení, jako jsou textové soubory nebo jednoduché databáze. Pokud známe délku projektu, daleko snáze se odhaduje objem dat, který bude potřeba uložit.

- **Zpracování a životní cyklus dat**

Je vhodné předem stanovit, jak budou data zpracovávána a jak dlouho mají být uchovávána. Ne všechny záznamy je nutné uchovávat trvale, některé lze po určité době agregovat, downsamplovat¹ nebo smazat. Definování jasné strategie mazání či archivace dat pomáhá předejít nekontrolovatelnému růstu databáze a zároveň usnadňuje správu a optimalizaci výkonu systému.

Tyto faktory pomohou určit, jestli je nutné ukládat neupravená data nebo předem zpracovaná.

Životnost dat

Při rozhodování o tom, jaké úložiště dat je vhodné, je klíčové porozumět životnímu cyklu dat. Čím realističtější budeme ohledně skutečného využití dat, tím méně jich je nutné ukládat a tím méně času strávíme hledáním ideálního úložného systému. Mnoho organizací zaznamenává příliš mnoho událostí a bojí se o ztrátu dat, ale mít obrovské množství špatně zpracovatelných dat je mnohem méně užitečné než mít agregovaná data uložená v relevantních časových intervalech.

U krátkodobých dat, například výkonnostních metrik, které se sledují jen kvůli ověření, že je vše v pořádku, možná vůbec nebude potřeba ukládat data v původní podobě, nebo alespoň ne na dlouho. To je zvláště důležité u dat založených na událostech, kde jednotlivé události samy o sobě nejsou významné a hlavní hodnotu mají až souhrnné statistiky. [2]

Předpokládejme, že sledujeme dobu načítání webové stránky v sekundách. V tabulce 2.1 je uvedena ukázka časové řady načítání webové stránky v různých časových okamžicích.

V takovém případě nás nezajímá každý jednotlivý záznam, ale spíše souhrnné statistiky, jako je průměrný čas načítání za minutu, hodinu nebo den. Proto můžeme zvolit strategii, kde budeme uchovávat pouze agregovaná data, například průměrný čas načítání za každou minutu, a původní záznamy po určité době smazat. Tímto způsobem můžeme výrazně snížit objem uložených dat a zároveň zachovat relevantní informace pro analýzu výkonu webové stránky. [2]

Výsledný záznam by tedy mohl vypadat například takto v tabulce 2.2.

¹*Downsampling* - proces převzorkování dat na menší počet datových bodů

Časové razítko	Čas načtení stránky
Duben 5, 2024 10:22:24	23s
Duben 5, 2024 10:22:28	15s
Duben 5, 2024 10:22:41	14s
Duben 5, 2024 10:23:02	11s
Duben 5, 2024 10:23:15	19s
Duben 5, 2024 10:23:33	12s

Tabulka 2.1.: Časová řada načítání webové stránky

Časové rozmezí	Prům. čas načtení	Počet přístupů	Min. čas	Max. čas
Duben 5, 2024 10:00:00	15.6s	6	11s	19s
Duben 5, 2024 11:00:00	17.2s	4	9s	25s

Tabulka 2.2.: Zjednodušený záznam časové řady načítání webové stránky

2.3. Databázové systémy

Databázové systémy jsou ten nejběžnější způsob jak ukládat časové řady. Většinou jsou připravené tzv. "out of the box", neboli ihned po pořízení, poskytují spoustu vestavěných funkcí pro jednoduché zpracování dat, jako je například agregace a filtrování. Navíc jsou optimalizované pro rychlý zápis a čtení dat, a dokáží škálovat s rostoucím objemem dat. [2]

SQL a NoSQL databáze

Hlavní rozdíl mezi SQL a NoSQL databázemi spočívá v jejich struktuře a způsobu ukládání dat. [[3]]

SQL SQL databáze jsou relační databáze, které používají strukturovaný dotazovací jazyk (SQL) pro správu a manipulaci s daty. Data jsou organizována do tabulek s pevně definovanými schématy, což znamená, že každá tabulka má předem stanovené sloupce a datové typy. Tento přístup je vhodný, když předem známe strukturu dat a víme, že se nebude měnit. [4]

id	item	qty
509	stul	8
510	zidle	16

Obrázek 2.2.: Ukázka struktury SQL databáze

NoSQL NoSQL databáze naopak nabízejí flexibilnější přístup k ukládání dat. Nejsou vázány na pevná schémata a umožňují ukládání dat v různých formátech, ty nejčastěji používané jsou:

- **Databáze párů klíč-hodnota**

Ukládají data jako páry klíč-hodnota, což umožňuje rychlý přístup k datům na základě klíče. Používá se v případech, kdy struktura klíčů je známá, ale hodnoty mohou být různého typu nebo formátu.

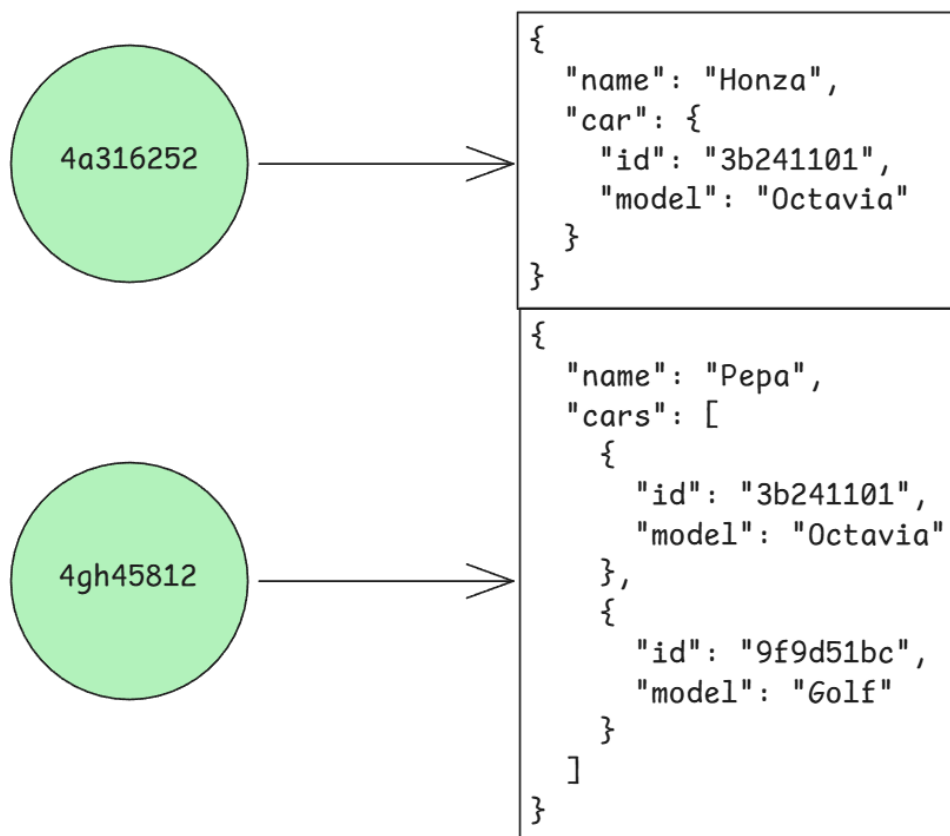
Jeden z nejznámějších příkladů je Redis, který je často využíván pro caching² a real-time aplikace jakožto dodatečná vrstva mezi uživatelem a koncovou databází. [5]

```
HSET 'user:001' first_name 'Jan' dob 12-JUN-1970
(integer) 2

HGETALL 'user:001'
'first_name '
'Jan '
'dob '
'12-JUN-1970 '
```

Ukázka kódu 2.1: Redis - ukázka syntaxe

²Caching - technika ukládání často používaných dat do rychlé paměti



Obrázek 2.3.: Ukázka struktury databáze párů klíč-hodnota

• Databáze dokumentů

Ukládají data ve formě dokumentů, často ve formátu JSON nebo BSON. Každý dokument může mít odlišnou strukturu, což umožňuje větší flexibilitu. Větší skupiny dokumentů mohou být organizovány do kolekcí. Podporují vnořené páry klíč-hodnota.

Nejznámější příklad je MongoDB. Často se používá pro webové aplikace, analytické systémy nebo logování, kde se struktura dat může často měnit. [6]

```

db.inventory.insertOne(
  {
    name: 'Jan',
    dob: new Date('1970-06-12')
  }
)

db.inventory.find({ name: 'Jan' }).pretty()
{
  _id: ObjectId('64f8e9a1b1234567890abcd1'),
  name: 'Jan',
  dob: ISODate('1970-06-12T00:00:00Z')
}

```

Ukázka kódu 2.2: MongoDB - ukázka syntaxe

- **Databáze sloupců**

Ukládají data podle sloupců namísto řádků, což je výhodné pro analytické dotazy, které často pracují s velkým množstvím dat v konkrétních sloupcích. Výhoda je v tom že to eliminuje potřebu čist nepotřebná data, což zvyšuje výkon dotazů.

Jeden z nejznámějších příkladů je ClickHouse, který je často využíván pro analytické úlohy v reálném čase, jako je sledování metrik a logování. [7]

```
INSERT INTO water_levels.sensor_readings (sensor_id, location,
      timestamp, water_level) VALUES
(201, 'River A', now(), 2.35);

SELECT sensor_id, location, timestamp, water_level
FROM water_levels.sensor_readings;
```

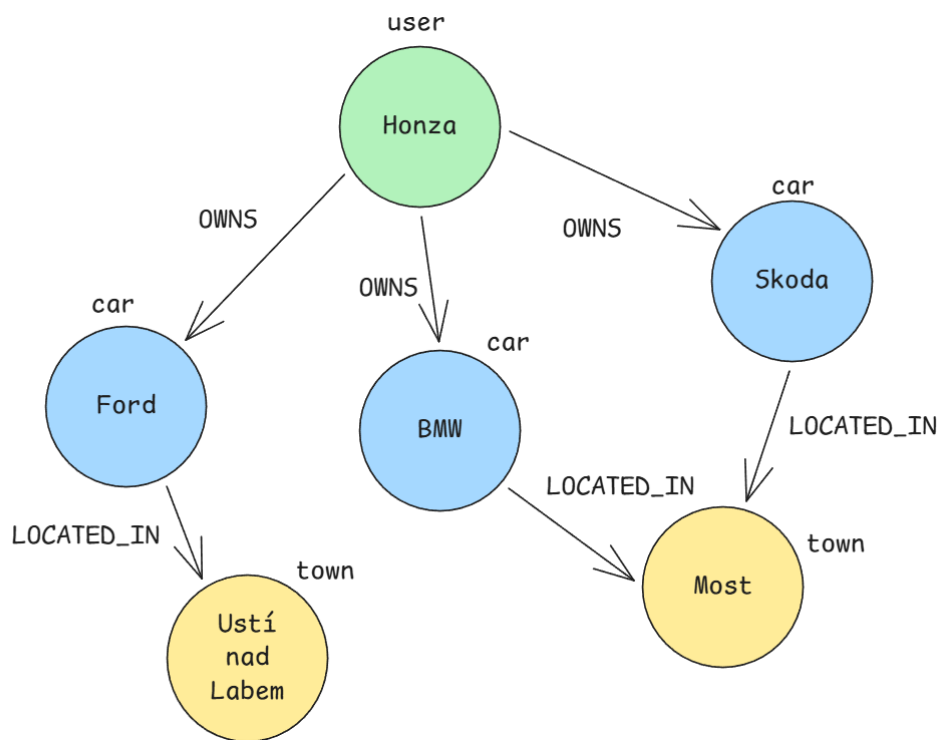
sensor_id	location	timestamp	water_level
201	River A	2025-11-18 12:34:56	2.35

Ukázka kódu 2.3: ClickHouse - ukázka syntaxe

- **Grafová databáze**

Ukládají data jako uzly a hrany, což je ideální pro reprezentaci a analýzu vztahů mezi daty. Uzly představují entity, zatímco hrany reprezentují vztahy mezi nimi.

Nejznámější příklad je Neo4j, který se často používá pro sociální sítě, doporučovací systémy a analýzu podvodů. [8]



Obrázek 2.4.: Ukázka struktury grafové databáze

```

CREATE (jan:Person {name: 'Jan'})
CREATE (eva:Person {name: 'Eva'})

MATCH (jan:Person {name: 'Jan'})
MATCH (eva:Person {name: 'Eva'})
CREATE (jan)-[:KNOWS]->(eva)

MATCH (jan:Person {name: 'Jan'})-[:KNOWS]->(friend)
RETURN jan.name AS Jan, friend.name AS Knows

Jan  | Knows
-----|-----
Jan  | Eva

```

Ukázka kódu 2.4: Neo4j - ukázka syntaxe

2.4. Úvod do problematiky testování a implementace

LXC

LXC je rozhraní uživatelského prostoru pro funkce jádra Linuxu umožňující izolaci procesů. Díky API a jednoduchým nástrojům umožňuje vytvářet a spravovat systémevé nebo aplikační kontejnery. [9]

LXD

LXD slouží jako alternativa k LXC a poskytuje uživatelsky přívětivější rozhraní pro správu kontejnerů. Interně ovšem také používá základní API LXC. Je doporučený hlavně pro méně zkušené uživatele.

Kontejner

Kontejner představuje standardizovanou softwarovou jednotku, která zapouzdřuje kód aplikace společně se všemi jejími závislostmi a knihovnami. Na rozdíl od virtuálních strojů kontejnery neobsahují vlastní operační systém a nevyužívají hypervizor. Místo toho sdílejí jádro hostitelského systému, což zajišťuje izolaci procesů při zachování minimální režie. Díky tomu jsou kontejnery výrazně menší (často v řádech megabajtů) a jejich spuštění je rychlejší. [10]

Klíčovou vlastností kontejnerů je jejich přenositelnost. Aplikaci zabalenou v kontejneru lze snadno přesouvat mezi různými prostředím – od lokálního vývoje až po produkční nasazení v cloudu – s jistotou, že bude fungovat konzistentně bez ohledu na podkladovou infrastrukturu. O masivní rozšíření a standardizaci této technologie se v posledních letech zasloužila především platforma Docker.

Virtuální stroj

Virtuální stroj je softwarová emulace fyzického počítače, která spouští vlastní kompletní operační systém v izolovaném prostředí. Provoz více virtuálních strojů na jednom fyzickém serveru umožňuje vrstva zvaná hypervizor, která se stará o přidělování hardwarových prostředků (CPU, paměť, úložiště) jednotlivým instancím. [10] Hypervizory lze rozdělit do dvou hlavních kategorií:

- **Bare-metal**

Běží přímo na fyzickém hardwaru hostitelského serveru bez prostřednictví operačního systému. Díky přímému přístupu k hardwarovým prostředkům je velmi efektivní a nabízí vysoký výkon, což z něj činí standardní volbu pro podniková datová centra a cloudová prostředí. Mezi známé zástupce patří VMware ESXi, Microsoft Hyper-V nebo KVM (Kernel-based Virtual Machine), který je součástí jádra Linuxu.

- **Hosted**

Instaluje jako běžná softwarová aplikace na již existující operační systém (tzv. hostitelský OS). Hardwarové zdroje jsou virtuálním strojům přidělovány prostřednictvím tohoto hostitelského systému, což zvyšuje režii a mírně snižuje výkon. Tento typ se nejčastěji využívá na osobních počítačích pro vývoj, testování nebo výuku. Typickými příklady jsou Oracle VirtualBox nebo VMware Workstation.

Ačkoliv jsou virtuální stroje náročnější na systémové zdroje než kontejnery, poskytují silnou úroveň izolace a umožňují provozovat aplikace vyžadující odlišné operační systémy na stejném

hardwaru. Historicky stály VM u zrodu cloud computingu a dodnes hrají klíčovou roli při konsolidaci serverů, optimalizaci využití hardwaru a v případech, kdy je vyžadována plná virtualizace prostředků.

2.5. Ansible

Ansible je open-source ³ nástroj pro automatizaci IT úloh, jako je konfigurace systémů, nasazení aplikací a správa infrastruktury. Na rozdíl od mnoha jiných automatizačních nástrojů nevyžaduje Ansible instalaci agentů ⁴ na cílové stroje – komunikuje s nimi typicky přes protokol SSH, což výrazně zjednodušuje jeho nasazení i údržbu. [11]

Ansible se připojí na cílový uzel přes určitý protokol a nahraje tam dočasný program, nazvaný **Ansible module**, který vykoná požadované úlohy a poté se sám odstraní.

Ansible byl vytvořen v roce 2012 Michaellem DeHaanem, známým také jako autorem nástrojů Cobbler a Func. Jeho cílem bylo nabídnout jednodušší a přístupnější alternativu k tehdejším nástrojům, které vyžadovaly složitější architekturu a používání agentů a zároveň poskytnout lepší možnost než jen jednorázový skript pro danou úlohu.

V roce 2015 Ansible získala společnost Red Hat, což přineslo projektu silnou podporu a další rozvoj. Pod vedením Red Hatu se Ansible stal jedním z nejpoužívanějších automatizačních nástrojů v podnikovém prostředí a stal se klíčovou součástí širší platformy Red Hat Automation. [11]

Idempotence

Ansible se snaží být idempotentní, což znamená, že opakované spuštění stejného playbooku by mělo vést k stejnému výsledku bez ohledu na to, kolikrát je spuštěn. Nė vždy je to ale možné, například při generování náhodných hesel nebo při práci s časově závislými daty.

Deklarativní přístup

Ansible umožňuje uživatelům popisovat cílový stav zařízení, nikoli jednotlivé kroky, jak tohoto stavu dosáhnout. Tyto kroky jsou v pozadí zajišťovány moduly. Může se ale stát, že uživatel bude chtít provést nějakou akci, pro kterou žádný modul neexistuje. V takovém případě je možné použít modul `command` nebo `shell`, kdy uživatel přímo specifikuje příkaz, který se má na cílovém uzlu spustit. [11]

³ *Open-source* - software, jehož zdrojový kód je volně dostupný pro uživatele k prohlížení, úpravám a distribuci.

⁴ *Agent* - proces instalovaný na cílovém uzlu, který přijímá instrukce od řídicího serveru, vykonává je a zpětně odesílá stav nebo výsledky

Čitelnost a jednoduchost

Konfigurace a úlohy jsou zapisovány v jazyce YAML, který je navržen tak, aby byl snadno čitelný a srozumitelný i pro ty, kteří nejsou odborníky na programování, tzv. sebedokumentující se jazyk. [11]

Základní architektura

- **Řídící uzel**

Řídící uzel je stroj, na kterém je nainstalován Ansible a odkud jsou spouštěny všechny automatizační úlohy. Obsahuje veškeré konfigurační soubory, playbooky a inventáře cílových uzlů. [12]

- **Řízené uzly**

Řízené uzly jsou servery nebo zařízení, která Ansible spravuje. Mohou to být například fyzické servery, virtuální stroje, kontejnery nebo síťová zařízení.

Ansible komunikuje s těmito uzly hlavně pomocí protokolu SSH a provádí na nich definované úlohy. Existují ale i jiné způsoby komunikace, například `local` pro lokální operace nebo `docker` pro správu Docker kontejnerů. [12]

V Ansible se dá připojit ať už k jednomu nebo více řízeným uzlům najednou. Pokud není specifikováno jinak, tak paralelně. Tyto skupiny jsou definovány v inventáři.

- **Úloha**

Úloha je základní jednotka práce v Ansible, která definuje konkrétní akci, kterou má Ansible provést na řízeném uzlu. Úlohy jsou definovány v playboocích a mohou zahrnovat různé operace, jako je instalace softwaru, kopírování souborů, spouštění příkazů nebo konfigurace služeb. [12]

```
- name: Ensure postgresql is at the latest version
  ansible.builtin.yum:
    name: postgresql
    state: latest
```

Ukázka kódu 2.5: Ansible - ukázka úlohy

- **Playbooky**

Playbooky jsou soubory ve formátu YAML, které obsahují sadu úloh pro Ansible. Definují, jaké úlohy mají být provedeny na kterých řízených uzlech a v jakém pořadí. Playbooky umožňují automatizovat složité procesy a zajišťují opakovatelnost a konzistenci konfigurací. [12]

```
---
- name: Update db servers
  hosts: dbservers
```

```
remote_user: root

tasks:
  - name: Ensure postgresql is at the latest version
    ansible.builtin.yum:
      name: postgresql
      state: latest

  - name: Ensure that postgresql is started
    ansible.builtin.service:
      name: postgresql
      state: started
```

Ukázka kódu 2.6: Ansible - ukázka playbooku

• **Moduly**

Moduly představují základní vykonávací jednotky v Ansible. Každá úloha (*task*) v playbooku využívá některý modul k provedení konkrétní operace na řízeném uzlu. Modul tedy definuje, jaký typ akce má být vykonán.

Ansible obsahuje velké množství vestavěných modulů, které jsou dostupné v tzv. *kolekcích*. Nejčastěji používané vestavěné moduly se nacházejí v kolekci `ansible.builtin`, například:

- `ansible.builtin.copy` – kopírování souborů mezi řídicím a řízeným uzlem,
- `ansible.builtin.user` – správa uživatelských účtů na řízeném uzlu,
- `ansible.builtin.service` – správa systémových služeb (start, stop, restart),

Lze najít a stáhnout i moduly z externích modulovacích kolekcí. [12]

• **Inventář**

Inventář je soubor, který obsahuje seznam řízených uzlů, které Ansible spravuje. Může být ve formátu INI, nebo YAML a buď statický, předem sestavený, nebo dynamický, vygenerovaný skriptem. Inventář umožňuje organizovat uzly do skupin, což usnadňuje cílení úloh na specifické sady zařízení. [12]

```
mail.example.com

[webservers]
foo.example.com
bar.example.com

[dbservers]
one.example.com
two.example.com
192.168.1.200
```

Ukázka kódu 2.7: Ansible - ukázka INI inventáře

```

ungrouped:
  hosts:
    mail.example.com:
webserver:
  hosts:
    foo.example.com:
    bar.example.com:
dbserver:
  hosts:
    one.example.com:
    two.example.com:
    192.168.1.200:

```

Ukázka kódu 2.8: Ansible - ukázka YAML inventáře

Ansible definuje vždy 2 hlavní seskupení v inventáři:

- **all** - obsahuje všechny zařízení v inventáři
- **ungrouped** - obsahuje zařízení, které nejsou přiřazeny do žádné skupiny

Všechny ostatní skupiny jsou definovány uživatelem a mohou být hierarchicky organizovány, což umožňuje vytvářet podskupiny a dědit vlastnosti mezi skupinami. [12]

• Role

Role jsou způsob, jak organizovat playbooky a úlohy do znovupoužitelných komponent. Takto vytvořené role mohou být dále znovu použity a také sdíleny přes Ansible Galaxy. Role umožňují strukturovat kód do předem definovaných adresářů, které vypadají následovně:

```

roles/
  common/
    tasks/
      main.yml
    handlers/
      main.yml
    templates/
      ntp.conf.j2
    files/
      bar.txt
      foo.sh
    vars/
      main.yml
    defaults/
      main.yml

```

```

    meta/
        main.yml
    library/
    module_utils/
    lookup_plugins/

    webtier/
    monitoring/
    fooapp/

```

Ukázka kódu 2.9: Struktura adresářů pro Ansible role

Jednotlivé adresáře v roli mají specifický účel:

- **tasks** - obsahuje hlavní úlohy, které se mají provést, obvykle v souboru `main.yml`
- **handlers** - obsahuje úlohy, které se spustí pouze v případě, že jsou "vyvolány" z jiných úloh, například pro restart služby po změně konfigurace
- **templates** - obsahuje šablony, které se používají pro generování konfiguračních souborů na základě proměnných
- **files** - obsahuje statické soubory, které se mají zkopírovat na řízené uzly
- **vars** - obsahuje proměnné, které se mají použít v souboru `main.yml`
- **defaults** - obsahuje proměnné, které se mají použít v souboru `main.yml`, ale s nižší prioritou než proměnné v adresáři `vars`
- **meta** - obsahuje metadata o roli, jako jsou závislosti na jiných rolích
- **library** - obsahuje vlastní moduly, které mohou být použity v úlohách
- **module_utils** - obsahuje pomocné funkce pro vlastní moduly
- **lookup_plugins** - obsahuje vlastní lookup pluginy, které umožňují získávat data z různých zdrojů

• Proměnné

Proměnné v Ansible umožňují ukládat hodnoty, které lze použít v různých částech playbooku. Tyto proměnné mohou být definovány na různých místech v projektu, a podle toho mají i přiřazené priority, pro případ kdyby se proměnné jmenovaly stejně (např. při definování výchozí cesty k souboru s konfigurací). [12]

Název proměnných je pouze omezen tím, že musí začínat písmenem nebo podtržítkem a může obsahovat pouze alfanumerické znaky a podtržítka. Dále jsou některé názvy proměnných rezervované, například `ansible_facts` nebo `inventory_hostname`. [12]

Proměnné se používají pomocí šablonovacího systému Jinja2, který umožňuje vkládat proměnné do textu a také provádět různé operace s nimi, jako jsou podmínky, smyčky nebo filtry. Aby Ansible věděl, že se jedná o proměnnou, musí být její název obklopen dvojími složenými závorkami a uzavřen v uvozovkách, například `"{{ promenna }}"`. [12]

```
- debug:
    msg: "Hodnota je {{promenna}}"
```

Ukázka kódu 2.10: Použití proměnné

– Fakta

Specifickým typem proměnných jsou tzv. "fakta", které Ansible automaticky shromažďuje o cílových uzlech během běhu playbooku.

Přístupuje se k nim přes proměnnou `ansible_facts` a obsahují informace o hardwaru, operačním systému, síťových rozhraních a dalších charakteristikách uzlu. K některým faktům lze přistupovat i zkráceně, například `ansible_facts['os_family']` lze zapsat jako `ansible_os_family`. [12]

```
{
  "ansible_all_ipv4_addresses": [
    "192.168.1.2"
  ],
  "ansible_all_ipv6_addresses": [
    "2345:0425:2CA1:0000:0000:0567:5673:23b5"
  ],
  "ansible_apparmor": {
    "status": "disabled"
  },
  "ansible_architecture": "x86_64",
  ...
}
```

Ukázka kódu 2.11: Shromážděná fakta

Shromažďování faktů lze ovlivnit pomocí proměnné `gather_facts`, která se nastavuje na úrovni playbooku. Jejich kolekce může pro některé případy být zbytečná a velmi zpomalovat průběh playbooku. [12]

Při kolizi názvů proměnných se použije následující neuplný výčet priority (vyšší číslo znamená vyšší prioritu):

1. Proměnné přidáné argumentem

```
ansible-playbook playbook.yml -e "promenna=foo"
```

Ukázka kódu 2.12: Proměnné přidáné argumentem

2. Proměnné role

Proměnné definované ve `roles/myrole/vars/main.yml`. Na stejné úrovni jsou i proměnné definované při zahrnutí role v playbooku.

```
- roles:
  - role: myrole
  vars:
    promenna: foo
```

Ukázka kódu 2.13: Proměnné při zahrnutí role

3. Registrované proměnné

```
- command: echo hello
register: promenna
```

Ukázka kódu 2.14: Registrované proměnné

Tyto proměnné jsou vytvořeny během běhu playbooku a obsahují výsledky z úloh, které je vytvořily. Využívají se například pro kontrolu úspěšného provedení úlohy nebo pro získání výstupu z příkazu.

4. Playbook proměnné

Proměnné definované přímo v souboru `playbook.yml`.

```
- hosts: all
  vars:
    promenna: foo
  tasks:
    ...
```

Ukázka kódu 2.15: Playbook proměnné

5. Host_vars proměnné

Proměnné definované pro konkrétní hosty v adresáři `host_vars/`. Každý soubor v tomto adresáři je pojmenován podle názvu hosta, pro který jsou proměnné určeny.

```
# host_vars/server1.yml
promenna: foo
```

Ukázka kódu 2.16: Proměnné hostů

6. Group_vars proměnné

Proměnné definované pro skupiny hostů v adresáři `group_vars/`. Každý soubor v tomto adresáři je pojmenován podle názvu skupiny, pro kterou jsou proměnné určeny.

```
# group_vars/web.yml
promenna: foo
```

Ukázka kódu 2.17: Proměnné skupin

7. Proměnné v inventáři

Proměnné definované přímo v inventářním souboru. Může se jednat o proměnné pro konkrétní hosty nebo pro celé skupiny podle umístění. Využívají se například pro definování specifických nastavení pro jednotlivé skupiny, jako je např. styl připojení.

```
[web]
server1 promenna=foo
```

Ukázka kódu 2.18: Proměnné v inventáři

8. Fakta

Automaticky shromážděné informace o cílových uzlech, přístupné přes proměnnou `ansible_facts`.

9. Výchozí proměnné role

Definované v `roles/myrole/defaults/main.yml`. Absolutně nejnižší priorita.

Citlivé proměnné, jako jsou hesla nebo klíče, by měly být šifrovány pomocí Ansible Vault, což je nástroj pro zabezpečení dat v Ansible.

Ten ale zabezpečuje data jen v případě "data at rest", to znamená uložená data v souborech. Jakmile jsou data během běhu playbooku dešifrována, nacházejí se v paměti a mohou být zobrazeny ve výstupu. Proto je vhodné citlivé informace skrýt pomocí parametru `no_log`, který zabrání jejich zobrazení ve výstupu playbooku. [12]

• Jinja2

Jinja2 je šablonovací jazyk pro Python, který Ansible používá pro zpracování proměnných a generování dynamického obsahu v playboocích.

Umožňuje vkládat proměnné do textu, provádět podmínky, smyčky a používat filtry pro manipulaci s daty. [13] **Příklad použití Jinja2 pro generování konfiguračního souboru z šablony.**

2.6. Monitorování

Prometheus

Prometheus je open-source systém pro monitorování a upozornění, který je navržen pro sběr a ukládání časových řad dat. Používá vlastní dotazovací jazyk PromQL pro analýzu a vizualizaci dat, což umožňuje uživatelům vytvářet komplexní dotazy a získávat hluboký náhled z monitorovaných systémů. [14]

Prometheus byl vyvinut společností SoundCloud v roce 2012 a od té doby se stal jedním z nejpopulárnějších nástrojů pro monitorování v cloudových a kontejnerizovaných prostředích. Nyní je celý projekt samostatnou organizací v rámci Cloud Native Computing Foundation (CNCF). [14]

Konfigurace

Prometheus se konfiguruje pomocí YAML souboru, kde se definují zdroje dat, pravidla pro shromažďování metrik a nastavení upozornění.

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - "alert.rules"

scrape_configs:
- job_name: prometheus
  static_configs:
  - targets: ['localhost:9090']
```

Ukázka kódu 2.19: Prometheus - ukázka konfigurace

Existují zde 3 hlavní bloky:

- **global** - obsahuje globální nastavení pro Prometheus, jako je interval shromažďování dat a interval vyhodnocování pravidel pro upozornění.
- **rule_files** - obsahuje seznam souborů s pravidly pro upozornění, které definují podmínky, za kterých má být odesláno upozornění.
- **scrape_configs** - obsahuje konfigurace pro shromažďování dat z různých zdrojů, včetně názvu úlohy a seznamu cílů, ze kterých se mají data shromažďovat.

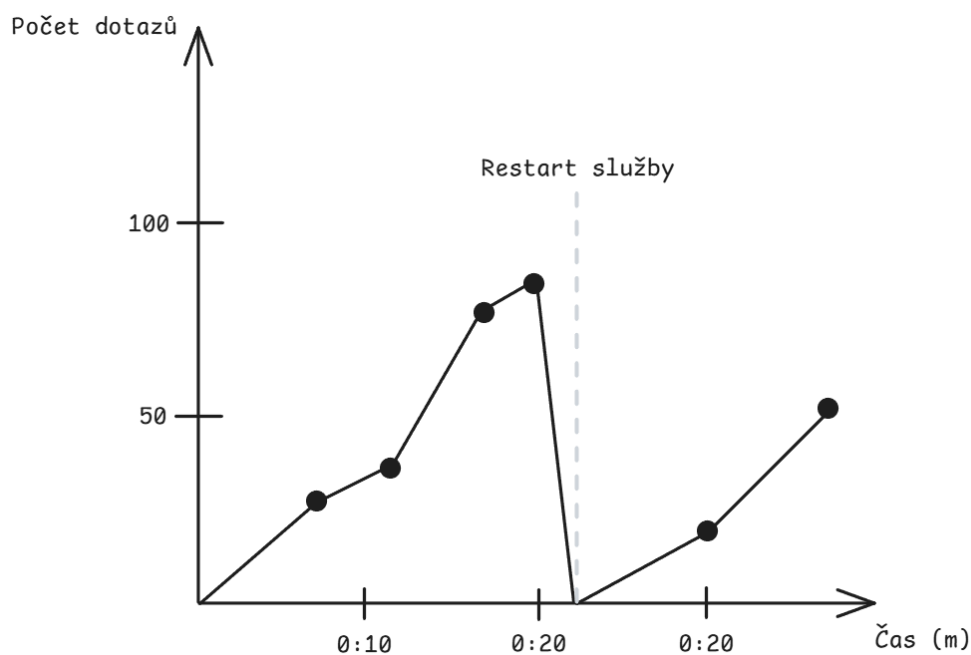
Datový Model

Prometheus ukládá data ve formě časových řad.

- **Metrika** - základní jednotka dat, která představuje konkrétní sadu měření, například `http_requests_total` pro počet HTTP požadavků.
- **Štítek** - klíč-hodnota pár, který poskytují další kontextové informace o metrice.
- **Hodnota** - konkrétní měření pro danou metriku a sadu štítků, například počet požadavků v určitém časovém okamžiku.

Dále se jednotlivé metriky dělí do 4 typů:

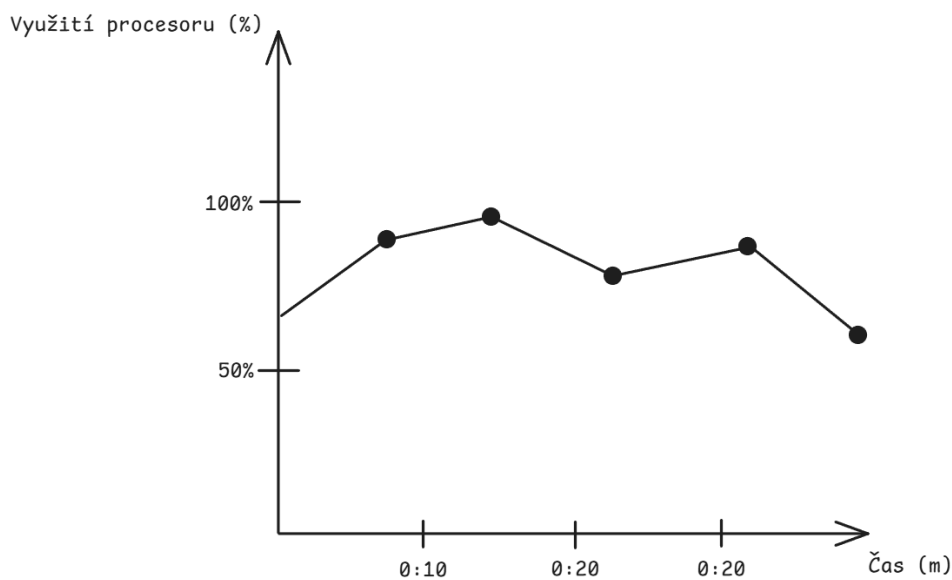
- **Counter**
Monotonicky rostoucí hodnota, která se zvyšuje pouze při události, například počet požadavků.



Obrázek 2.5.: Ukázka metriky typu Counter

- **Gauge**

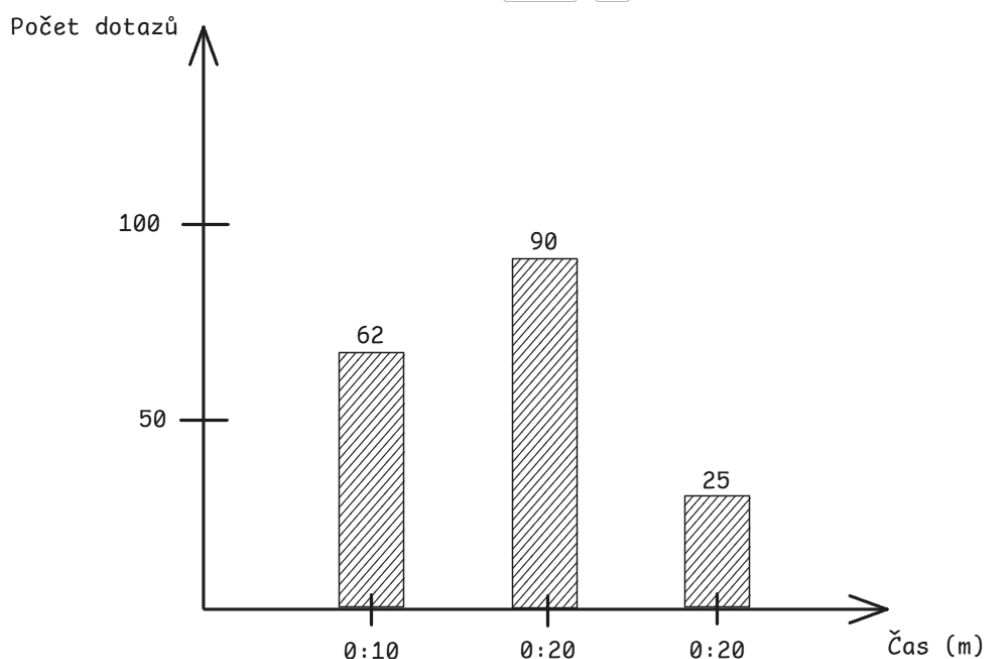
Hodnota, která může růst i klesat, například aktuální využití CPU.



Obrázek 2.6.: Ukázka metriky typu Gauge

- **Histogram**

Měří distribuci hodnot, například doby odezvy, a poskytuje informace o počtu vzorků a součtech pro různé intervaly.



Obrázek 2.7.: Ukázka metriky typu Histogram

- **Summary**

Podobný histogramu, ale poskytuje kvantily pro měření distribuce, například 95. percentil doby odezvy.

PromQL

PromQL, celým názvem Prometheus Query Language, je dotazovací jazyk pro Prometheus. Dotazy mohou být buď "instantní", které vracejí aktuální hodnoty metrik, nebo "časový rozsah", které vracejí historické hodnoty pro zadané časové období. [14]

```
rate(http_requests_total[5m])[30m:1m]
```

Ukázka kódu 2.20: PromQL - ukázka dotazů

Tento dotaz vrací rychlost změny metriky `http_requests_total` za posledních 5 minut, přičemž se tato rychlost počítá pro každou minutu v posledních 30 minutách.

Node Exporter

Node Exporter je nástroj pro sběr systémových metrik z hostitelského systému, který je monitorován pomocí Prometheus. Node Exporter shromažďuje informace o specifickém systému pro který je konfigurován, například o CPU, paměti, disku, síti a dalších hardwarových a systémových parametrech. [14]

Je třeba ho distribuovat a spustit na každém hostitelském systému, který chceme monitorovat, a poté nakonfigurovat Prometheus, aby shromažďoval data z těchto Node Exporterů. [14]

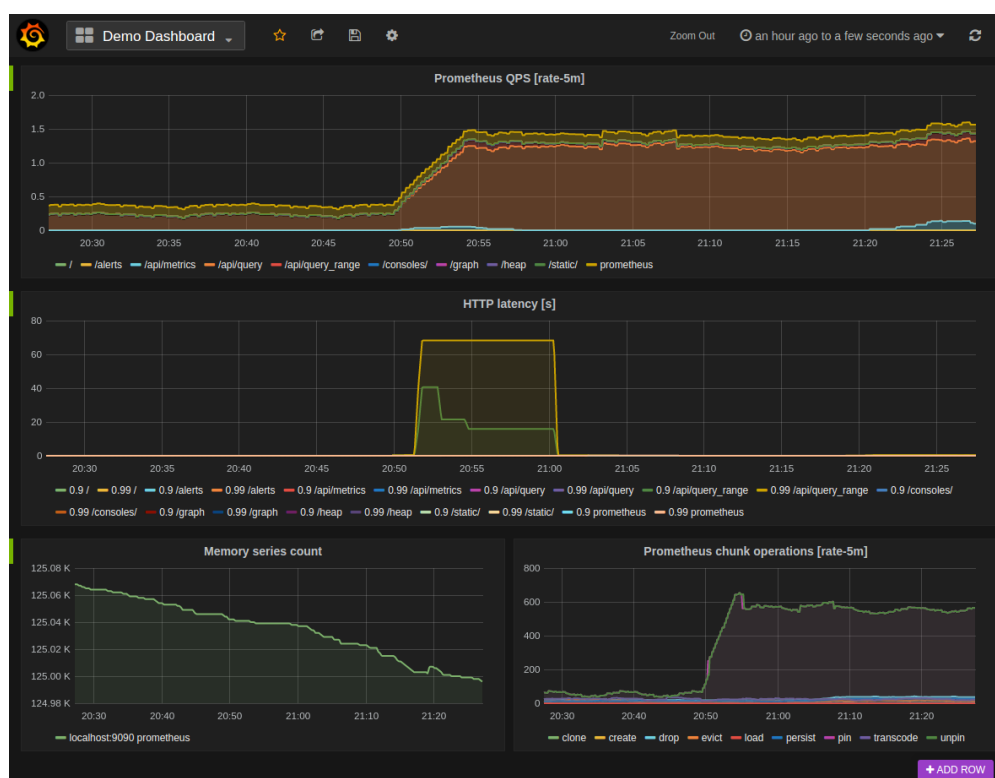
Alertmanager

Alertmanager je komponenta Prometheus, která se stará o správu a odesílání upozornění. Umožňuje definovat pravidla pro upozornění, která se spouštějí na základě dotazů PromQL, a poté odesílat upozornění prostřednictvím různých kanálů, jako jsou e-mail, Slack a další. [14]

Grafana

Grafana je open-source platforma pro vizualizaci a analýzu dat, která umožňuje vytvářet interaktivní dashboardy a grafy z různých datových zdrojů. Grafana podporuje širokou škálu datových zdrojů, včetně databází, časových řad, cloudových služeb a dalších, což umožňuje uživatelům snadno integrovat a vizualizovat data z různých systémů. [15]

Grafana byla původně vyvinuta v roce 2014 Torkel Ödegaardem jako větev projektu Kibana. Nyní je Grafana Labs, společnost založená v roce 2015, hlavním vývojářem a správcem projektu.



Obrázek 2.8.: Ukázka dashboardu v Grafaně [15]

Grafana poskytuje vestavěnou podporu pro širokou škálu datových zdrojů, jako jsou například Prometheus, InfluxDB, Grafana Loki, SQL databáze a další. [15] Dále je tu možnost přidat i vlastní datové zdroje pomocí pluginů. [15]

Vizualizace

Než může Grafana data vizualizovat, musí být nejprve dotazována na data z připojených datových zdrojů. Na tyto zdroje se připojí pomocí tzv. pluginů, které umožňují jak komunikaci, tak konvert

dat z příchozího formátu (JSON, CSV...) do formátu, který Grafana dokáže zpracovat. Takový formát se nazývá *data frame*. [15]

Interní struktura takového data framu vypadá jako tabulka, kde každý sloupec představuje jednu proměnnou a každý řádek představuje jeden záznam.

time	temperature
2024-05-05 10:22:24	43.0
2024-05-05 10:22:25	45.3
2024-05-05 10:22:26	44.4
2024-05-05 10:22:27	41.2
2024-05-05 10:22:28	49.7
2024-05-05 10:22:29	67.0

Tabulka 2.3.: Grafana - ukázka data framu teploty CPU

Pokud by pro různá data bylo časové razítko stejné, Grafana je dokáže sloučit do jednoho data framu, což umožňuje například porovnávat různé metriky v čase. [15]

Následně se vytvoří dotazy, které vybírají konkrétní data z data framu. Tyto dotazy jsou psány v jazyce specifickém pro daný datový zdroj, například PromQL pro Prometheus. [15] Nakonec se z dat vrácených dotazem vytvoří panel. Panel je kontejner pro vizualizaci dat, který může být různých typů, například graf, tabulka, text nebo mapa. [15] Každý panel může být nakonfigurován tak, aby zobrazoval data určitým způsobem, například pomocí různých barev, os nebo legend. [15]

2.7. Testování

Testování výkonu a spolehlivosti softwarových systémů představuje nedílnou součást návrhu, vývoje a provozu moderních aplikací. Jeho hlavním cílem je systematicky ověřit chování systému při různých úrovních a typech zátěže, identifikovat úzká hrdla, vyhodnotit stabilitu a zajistit, aby systém splňoval požadavky kladené reálným provozem.

V testování můžeme definovat několik klíčových odvětví:

- **Spolehlivost**

Typickými příklady spolehlivosti jsou:

- aplikace funguje tak, jak uživatel očekává
- dokáže tolerovat chyby uživatele
- výkon je dostatečně dobrý pro vyžadované použití, pod očekávaným zatížením
- dokáže zabránit neoprávněnému přístupu a využívání

Pokud jsou všechny tyto body splněny, můžeme říci, že systém je spolehlivý. [16]

Kvalitním nástrojem jak testovat spolehlivost jsou například chaos testy, které záměrně způsobují poruchy v systému, aby se ověřilo, zda dokáže správně reagovat a zotavit se z nich. Jedním takovým nástrojem je Chaos Monkey od Netflixu, který náhodně vypíná instance v produkčním prostředí, aby otestoval odolnost systému vůči selhání. [17]

• Škálovatelnost

Škálovatelnost se týká schopnosti systému růst nebo zmenšovat své zdroje v závislosti na zátěži. Systém musí být schopen efektivně zpracovávat rostoucí množství požadavků bez výrazného snížení výkonu.

Zátěž je ovšem ale více než jen počet požadavků. Může zahrnovat i různé typy zátěže, jako jsou například:

- dotazy za sekundu
- poměr čtení a zápisu
- současní aktivní uživatelé

Také je důležité identifikovat, kde se nacházejí úzká hrdla systému, která mohou omezovat jeho škálovatelnost. V jednom systému mohou být problém celkové průměrné zvednutí uživatelů, ale v jiném systému může být extrémní nárůst uživatelů v určité období - odlehlé hodnoty. [16]

• Výkon

V oblasti výkonu jsou dvě klíčové otázky, které je třeba zodpovědět:

- Když se zvedne zátěž, ale systémové zdroje zůstávají stejné, jak to ovlivní výkon?
- Když se zvedne zátěž, o kolik musíme zvednout systémové zdroje, aby výkon zůstal stejný?

V systémech které zpracovávají data se často bavíme o propustnosti, která udává, kolik dat je systém schopen zpracovat za jednotku času. V online systémech zase o výkonu často mluvíme v souvislosti s latencí, která udává, jak dlouho trvá zpracování jednotlivého požadavku. [16]

Databázových systémů se týkají všechny tyto aspekty, a proto je důležité je testovat a porovnávat, aby bylo možné vybrat ten nejvhodnější pro konkrétní použití.

Benchmarking a porovnávání systémů

Benchmarking je metodický přístup k porovnávání výkonnosti různých systémů za definovaných a reprodukovatelných podmínek. Cílem je zajistit objektivní srovnatelnost výsledků a eliminovat vlivy okolního prostředí, které by mohly měření zkreslit.

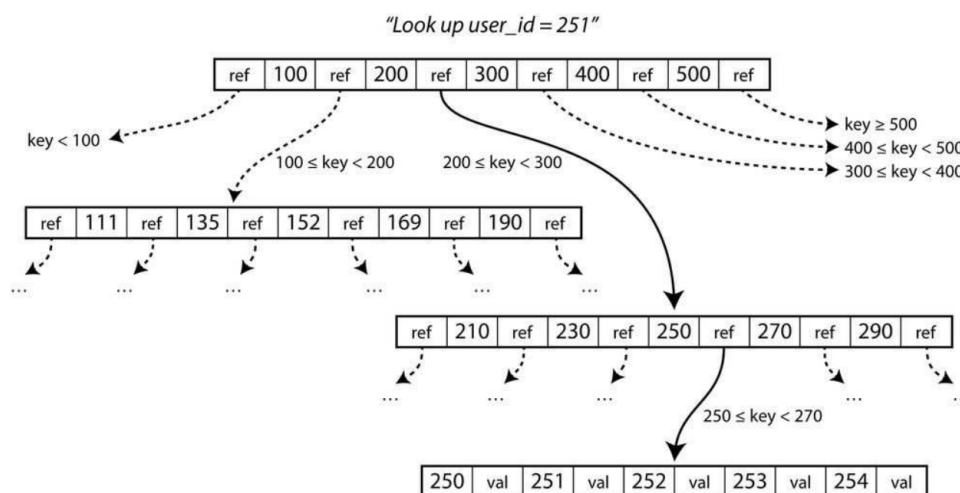
V oblasti databázových systémů se benchmarking využívá například pro porovnání různých datových struktur a úložných strategií, jako jsou B-Tree a LSM stromy. [16].

Klíčovým faktorem benchmarkingu je definice workloadu, který musí odpovídat reálnému použití systému. Nevhodně zvolený scénář může vést k výsledkům, které neodrážejí skutečné chování aplikace.

B-Tree

B-Tree je datová struktura, která uchovává data ve formě klíč-hodnota, která jsou seřazené podle klíče.

B-Tree rozdělí úložiště dat do několika fixních bloků (často odpovídajících velikosti diskového sektoru). Každý takový blok může být lehce identifikovaný pomocí adresy a jeden blok může více adres odkazujících na jiné bloky. [16] Jeden blok je označený za kořenový blok, který je vždy výchozím bodem pro hledání dat.



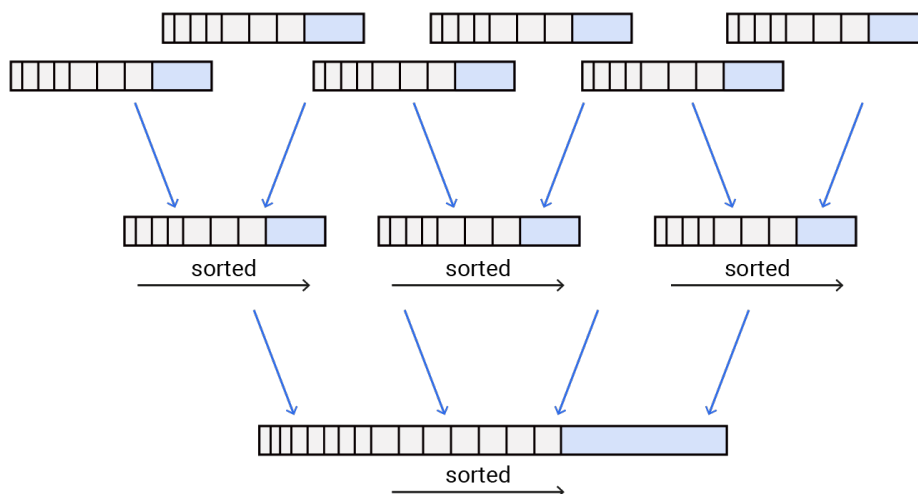
Obrázek 2.9.: Ukázka organizace dat v B-Tree struktuře [16]

V ukázce na obrázku 2.9 hledáme záznam s klíčem 251, takže víme, že musíme sledovat odkaz do seznamu 200-299. Takový seznam se dále dělí do bloků seřazené od x-209, 210-229, 230-249, 250-269, 270-289 a 290-x. Eventuelně se dostaneme až do správného bloku který hledáme. [16]

LSM Strom

Log-Structured Merge-Tree je datová struktura optimalizovaná pro vysokou propustnost zápisů. Každý zápis se nejprve uloží do rychlé paměti (memtable) a periodicky se přepíše do trvalého úložiště (disku) ve formě seřazených souborů (SSTables). Tyto SSTables jsou přirozeně seřazené podle klíče a je už dále neměnný. [16]

Při čtení se pak postupuje nejdříve přečtením memtables a následně od nejnovějších SSTables k nejdřívějším, dokud se nenajde požadovaný záznam nebo se nedojde na konec. [16]



Obrázek 2.10.: Ukázka organizace dat v LSM stromu [18]

Starší SSTables jsou pravidelně slučovány do větších souborů, aby se snížil počet souborů, které je třeba prohledávat při čtení, a aby se odstranily záznamy, které byly aktualizovány nebo smazány. [16] Díky tomu že jsou přirozeně seřazené, je tento proces slučování efektivní. [16]

3. Praktická část

3.1. Popis dat

Poskytovatelem dat je Datové centrum Ústeckého kraje (DCUK), které spravuje a zpřístupňuje data z různých systémů veřejné správy. Použitý dataset má velikost přibližně 650 MB a obsahuje zhruba 600 tisíc záznamů. Data jsou ve strukturovaném formátu CSV. Jedná se o záznamy ze systému pro sledování pohybu vozidel správy a údržby silnic. Každý záznam obsahuje zejména identifikátor vozidla, typ aktivity a geografickou polohu. Tyto informace umožňují analýzu pohybu vozidel v čase a prostoru, například pro vyhodnocení efektivity údržby silniční sítě nebo sledování tras jednotlivých vozidel.

3.2. Předběžná srovnávací analýza systémů

ClickHouse

ClickHouse je sloupcově orientovaná databáze určená pro analytické dotazy v reálném čase (OLAP), původně vyvinutá společností Yandex jako interní nástroj pro zpracování velkých objemů dat. Později byla uvolněna jako open-source projekt a postupně se stala jednou z populárních databází pro analytické zpracování dat. [19] Hlavním důvodem vzniku ClickHouse byla potřeba efektivního zpracování a analýzy rozsáhlých logových a telemetrických dat v reálném čase, kde tradiční relační databáze nedokázaly splnit požadavky na výkon.

ClickHouse je dostupný jak ve formě open-source řešení pro self-hosting, tak i jako plně spravovaná cloudová služba. Open-source varianta umožňuje nasazení na vlastních serverech nebo v libovolném cloudovém prostředí.

MariaDB ColumnStore

MariaDB ColumnStore je sloupcové rozšíření relační databáze MariaDB zaměřené na analytické zpracování velkých objemů dat.

ColumnStore umožňuje distribuované zpracování dat a paralelizaci dotazů, což výrazně zlepšuje výkon při práci s rozsáhlými datasety. Je vhodný zejména pro analytické reporty a agregace nad velkými tabulkami. [20]

InfluxDB

InfluxDB je časově orientovaná databáze navržená pro ukládání a analýzu dat závislých na čase, například metrik, senzorických dat nebo logů.

Vznikla v roce 2013 společností InfluxData jako odpověď na rostoucí potřebu efektivního ukládání a dotazování časových řad. Je optimalizována pro vysokou rychlost zápisu a práci s časovými intervaly, což ji činí vhodnou pro IoT, monitoring a observabilitu systémů. [21]

IoTDB

IoTDB je časově orientovaná databáze vyvinutá Apache Software Foundation zaměřená na IoT scénáře a edge computing.

Byla navržena pro efektivní ukládání a zpracování velkého množství senzorických dat s vysokou frekvencí zápisu. Podporuje hierarchickou strukturu dat a je optimalizována pro nízké nároky na úložiště i výpočetní výkon. [22]

MariaDB

MariaDB je relační databázový systém vzniklý jako fork MySQL po jeho převzetí společností Oracle v roce 2009. Cílem projektu bylo zachovat open-source charakter a komunitní vývoj.

MariaDB je široce používaná pro transakční systémy, webové aplikace a běžné ukládání strukturovaných dat. Nabízí vysokou kompatibilitu s MySQL a stabilní výkon pro klasické databázové operace. [23]

MongoDB

MongoDB je dokumentově orientovaná NoSQL databáze, která ukládá data ve formátu BSON (binární JSON). Byla navržena pro flexibilní práci se schématy a snadnou škálovatelnost.

Je vhodná pro aplikace, kde se struktura dat často mění, například webové služby, katalogy nebo IoT aplikace. [24]

QuestDB

QuestDB je vysoce výkonná open-source time-series databáze zaměřená na nízkou latenci a vysokou propustnost při zpracování dat v reálném čase.

Byla navržena s důrazem na jednoduchost, SQL kompatibilitu a extrémně rychlé dotazy nad časovými řadami. Využívá se zejména v oblasti finančních dat, monitoringu a IoT. [25]

TimescaleDB

TimescaleDB je rozšíření relační databáze PostgreSQL zaměřené na práci s časovými řadami. Kombinuje klasický SQL přístup s optimalizacemi pro time-series data.

Vznikla v roce 2017 jako open-source projekt s cílem propojit výhody relačního modelu a specializovaných time-series databází. Je vhodná pro monitoring, IoT a analytické systémy, kde je potřeba jak flexibilita SQL, tak efektivní práce s časem. [26]

Dopsat informace k jednotlivým databázím

3.3. Návrh testovacího scénáře

Testovací prostředí je postaveno na virtualizační technologii LXD, která umožňuje vytváření izolovaných kontejnerů s nízkou režijní náročností. Každý testovaný databázový systém běží ve svém samostatném LXD kontejneru, čímž je zajištěna oddělenost prostředí a reprodukovatelnost výsledků. Všechny kontejnery mají přidělené stejné hardwarové prostředky, aby byly výsledky jednotlivých testů mezi databázemi porovnatelné.

Proces vytváření kontejnerů, instalace databázových systémů a spuštění testů je plně automatizován pomocí nástroje Ansible. Tím je zajištěna reprodukovatelnost testovacího prostředí napříč všemi testy a minimalizace lidského zásahu.

Struktura Ansible projektu je navržena modulárně:

- **Vytvoření kontejneru**

Pomocí Ansible modulu `community.general.lxd_container` je vytvořen nový LXD kontejner se specifikovanými parametry.

- **Instalace databázového systému**

Po vytvoření kontejneru následuje instalace zvolného databázového systému podle odpovídající Ansible role. Každý systém má vlastní roli, která zajišťuje instalaci, konfiguraci a spuštění služby.

- **Příprava na testování**

Po instalaci databáze je kontejner nakonfigurován pro testování. Nahrávají se testovací skripty, dataset a konfigurační soubory.

Cílem testovacího scénáře je porovnat vybrané databázové systémy z hlediska výkonu ve vybraných testech. Testy budou zaměřeny na následující oblasti:

- Rychlost jednorázového zápisu dat
- Vrácení jednoho specifického záznamu
- Vrácení všech záznamů za určitý časový interval
- Vrácení všech záznamů splňujících určitou podmínku

- Agregáčn  dotazy
- Rychlost z pisu dat v re ln m  ase

B hem v ech test  se budou sb rat metriky jako doba trv n  dotazu, vyu it  CPU, vyu it  pam ti a I/O operace disku. V echny tyto metriky budou n sledn  zaps ny p es Node Exportery v ka d m virtu ln m stroji. Ty budou n sledn  d ny k dispozici slu b  Prometheus, která bude zaji  ovat sb r a uchov v n  t chto metrik pro n slednou anal zu a vizualizaci Grafanou.

Ve ker  testy se budou prov d t na stejn m hardwarov m vybaven  a za stejn ch podm nek, na virtu ln ch stroj  s p edem definovan mi parametry (CPU, RAM, diskov  prostor).

3.4. Konfigurace testovac ho prost ed  a datab zov ch syst m 

Virtualizace

C lem bylo vytvo it testovac  prost ed , kter  bude co nejv ice izolovan  a z roveň umo n  snadnou spr vu a automatizaci. Z t chto d vod  jsem si zvolil virtualiza n  technologii LXD, kter  umo  uje vytv r n  a spr vu kontejner  a virtu ln ch stroj .

Zvolil jsem si virtu ln  stroje pro jejich v t  i izolaci a lep  i kontrolu nad prost ed m, ve kter m budou testy prob hat. To zajist ,  e v sledky test  budou co nejv ice objektivn  a porovnateln  mezi jednotliv mi datab zov mi syst my.

Dal   v hodou tohoto p stupu je mo nost snadno resetovat testovac  prost ed  mezi jednotliv mi testy, co  zaji  uje konzistenci a reprodukovatelnost v sledk .

Architektura

Ka d  testovan  datab zov  syst m b  i ve sv m samostatn m LXD kontejneru, kter  je nakonfigurov n s p r d len mi hardwarov mi prost edky (CPU, RAM, diskov  prostor).

P i vytvo en  kontejneru se do n   nainstaluje zvolen  datab zov  syst m a Node Exporter pro sb r metrik.

Na vyhrazen m virtu ln m zdroji b  i Grafana a Prometheus, kter  zaji  uj  sb r a vizualizaci metrik z jednotliv ch kontejner .

Ansible

```
root/  
|  
|-- ansible.cfg
```

```

|-- setup.sh
|-- run_test.sh
|-- run_live_insert.sh
|
|-- setup/
|   |-- setup.yml
|   |-- setup_monitoring.yml
|   \-- reload_prometheus.yml
|
|-- vars/
|   |-- resource_profiles.yml
|   \-- test_types.yml
|
|-- roles/
|   |-- common-create/
|   |-- common-start/
|   |-- common-post-processing/
|   |-- profile-loader/
|   |-- node-exporter/
|   |-- monitoring/
|   |   |-- tasks/
|   |   |-- templates/
|   |   \-- files/
|   |
|   |-- clickhousedb/
|   |   |-- tasks/
|   |   |   |-- install.yml
|   |   |   \-- setup_database.yml
|   |   \-- defaults/
|   |       \-- main.yml
|   |
|   |-- ...
|   \-- columnstoredb/
|
\-- tests/
    |-- clickhousedb/
    |   |-- insert.yml
    |   |-- aggregate.yml
    |   |-- condition.yml
    |   |-- single_point.yml
    |   \-- live_insert.yml
    |
    |-- ...

```

```
| -- columnstoredb/  
\ -- generate_live_data.py
```

Ukázka kódu 3.1: Struktura adresářů pro Ansible

Projekt se skládá z několika klíčových adresářů a souborů:

- **inventory/**

Inventář obsahuje definice hostů, databázových systémů, které mají přednastavenou IP adresu.

- **roles/**

Role v projektu bych rozdělil do 3 kategorií:

- **common**

Obsahuje role pro vytváření kontejnerů, jejich spuštění a post-processing výsledků.

- **monitoring**

Role pro instalaci a konfiguraci Node Exporterů, Prometheus a Grafany.

- **databases**

Každý testovaný databázový systém má svou vlastní roli, která obsahuje úkoly pro instalaci, konfiguraci a spuštění služby. Dále obsahuje i datový soubor, který následně testovací skripty použijí pro naplnění databáze testovacím datasetem.

- **tests/**

Adresář obsahuje definice testů pro jednotlivé databázové systémy.

- **setup/**

Adresář obsahuje ansible playbooku, které slouží jako vstupní body pro vytvoření testovacího prostředí a nastavení monitorovacího systému.

Konfigurace databází

3.5. Pilotní nasazení systémů

Pilotní testování bylo provedeno na mém osobním notebooku, který má následující hardwarové parametry:

- AMD Ryzen™ 5 5600H (max. zvýšený takt 4,2 GHz, 16 MB mezipaměti L3, 6 jader, 12 vláken)
- 16GB paměť DDR4-3200 MHz RAM (2x 8 GB)
- NVIDIA® GeForce® GTX 1650 (4 GB vyhrazené paměti GDDR6)
- Disková jednotka SSD 512 GB PCIe® NVMe™ M.2
- Integrovaná síťová karta 10/100/1000 GbE LAN

Na notebooku je nainstalován operační systém Ubuntu 22.04 LTS v minimální (serverové) konfiguraci bez grafického uživatelského rozhraní. Systém je ovládán výhradně prostřednictvím příkazové řádky (CLI). Minimální konfiguraci jsem zvolil z důvodu optimalizace výkonu a minimalizace spotřeby systémových prostředků, které by mohly ovlivnit výsledky testů.

3.6. Implementace testů

Pro testování databázových systémů jsem následující sadu testů:

- **Insert test**

Test zaměřený na měření rychlosti zápisu dat do databáze.

- **Single point test**

Test zaměřený na měření rychlosti vrácení jednoho specifického záznamu z databáze.

- **Condition test**

Test zaměřený na měření rychlosti vrácení všech záznamů splňujících určitou podmínku. Jako podmínku jsem si zvolil vrácení všech záznamů, které mají polohu v určitém geografickém rozsahu za určité období.

- **Aggregate test**

Test zaměřený na měření rychlosti provedení agregačního dotazu nad databází. V agregačním testu jsem zvolil dotaz, který vrací průměrnou rychlost, maximální rychlost a počet záznamů pro každé vozidlo za určité období.

Všechny tyto testy se spouští pomocí skriptu `run_test.sh`, který přijímá parametry pro výběr databázového systému a typu testu.

```
./run_test.sh -d clickhousedb -t insert
```

Ukázka kódu 3.2: Příklad spuštění live insert testu

- **Live insert test**

Test zaměřený na měření rychlosti zápisu dat v reálném čase, kdy se do databáze kontinuálně vkládají nové záznamy. Má své speciální parametry:

- **workers**

Počet paralelních zdrojů, které vkládají data do databáze.

- **duration**

Celková doba v sekundách, po kterou bude test probíhat.

- **batch_size**

Počet záznamů v jedné dávce, která bude vložena do databáze najednou.

Test se zavolá následujícím způsobem:

```
./run_live_insert.sh -d clickhousedb -w 4 -t 300 -b 1000
```

Ukázka kódu 3.3: Příklad spuštění live insert testu

Příkaz nastaví test pro databázový systém ClickHouse přes parametr `-d`, počet zdrojů na 4 přes parametr `-w`, dobu trvání testu na 300 sekund (5 minut) přes parametr `-t` a velikost dávky na 1000 záznamů přes parametr `-b`.

Ze všech testů se sbírají metriky dvojím způsobem:

- **NDJSON výsledky**

Po dokončení každého testu se výsledky ukládají do souboru ve formátu NDJSON (Newline Delimited JSON), který obsahuje informace o době trvání dotazu, počtu záznamů vrácených dotazem...

- **Prometheus metriky**

Během testů se v každém kontejneru sbírají metriky pomocí Node Exporteru, které jsou následně shromažďovány službou Prometheus. Tyto metriky zahrnují využití CPU, využití paměti, I/O operace disku a síťovou aktivitu.

4. Výsledky výzkumu a diskuse

5. Závěr

Seznam použitých zdrojů

1. DIGGLE, Peter; GIORGI, Emanuele. *Time Series: A Biostatistical Introduction*. 2. vyd. OUP Oxford, 2025. Oxford Statistical Science Series. ISBN 9780191024207. Dostupné také z: <https://books.google.cz/books?id=09s2EQAAQBAJ>.
2. NIELSEN, Aileen. *Practical time series analysis: Prediction with statistics and machine learning*. O'Reilly Media, 2019. ISBN 9781492041658. Dostupné také z: <https://books.google.cz/books?id=odCwDwAAQBAJ>.
3. MICROSOFT. *What is NoSQL database?* [online]. [cit. 2025-11-10]. Dostupné z: <https://azure.microsoft.com/cs-cz/resources/cloud-computing-dictionary/what-is-nosql-database>. Online. Microsoft.
4. MICROSOFT. *What is SQL database?* [online]. [cit. 2025-11-10]. Dostupné z: <https://azure.microsoft.com/cs-cz/resources/cloud-computing-dictionary/what-is-sql-database>. Online. Microsoft.
5. REDIS. *Use Cases | Docs* [online]. [cit. 2025-11-10]. Dostupné z: https://redis.io/docs/latest/develop/data-types/timeseries/use_cases/. Online. Redis.
6. MONGODB. *Use Cases | MongoDB* [online]. [cit. 2025-11-10]. Dostupné z: <https://www.mongodb.com/solutions/use-cases>. Online. MongoDB.
7. CLICKHOUSE. *Use Cases | ClickHouse* [online]. [cit. 2025-11-10]. Dostupné z: <https://clickhouse.com/use-cases>. Online. ClickHouse.
8. NEO4J. *Graph Database Use Cases & Solutions* [online]. [cit. 2025-11-10]. Dostupné z: <https://neo4j.com/use-cases/>. Online. Neo4j.
9. CONTAINERS, Linux. *Linux Containers - LXC - Introduction* [online]. [cit. 2025-11-10]. Dostupné z: <https://linuxcontainers.org/lxc/introduction/>. Online. Linux Containers.
10. HAT, Red. *Containers vs VMs* [online]. [cit. 2025-11-10]. Dostupné z: <https://www.redhat.com/en/topics/containers/containers-vs-vms>. Online. Red Hat.
11. HOCHSTEIN, Lorin; MOSER, Rene. *Ansible: Up and Running: Automating Configuration Management and Deployment the Easy Way*. O'Reilly Media, 2017. ISBN 9781491979778. Dostupné také z: <https://books.google.cz/books?id=h5YtDwAAQBAJ>.
12. HAT, Red. *Ansible Community | Ansible documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://docs.ansible.com>. Online. Red Hat.

13. JINJA2. *Jinja2 Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://jinja.palletsprojects.com/>. Online. Jinja2.
14. PROMETHEUS. *Prometheus Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://prometheus.io/docs/>. Online. Prometheus.
15. LABS, Grafana. *Grafana Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://grafana.com/docs/>. Online. Grafana Labs.
16. KLEPPMANN, Martin. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017. ISBN 9781491903100. Dostupné také z: <https://books.google.cz/books?id=p1heDgAAQBAJ>.
17. MONKEY, Chaos. *Chaos Monkey* [online]. [cit. 2025-11-10]. Dostupné z: <https://netflix.github.io/chaosmonkey/>. Online. Chaos Monkey.
18. SCYLLADB. *Log-Structured Merge-Tree Definition* [online]. [cit. 2025-11-10]. Dostupné z: <https://www.scylladb.com/glossary/log-structured-merge-tree/>. Online. ScyllaDB.
19. CLICKHOUSE. *What is ClickHouse?* [online]. [cit. 2025-11-10]. Dostupné z: <https://clickhouse.com/docs/intro>. Online. ClickHouse.
20. MARIADB. *MariaDB ColumnStore* [online]. [cit. 2025-11-10]. Dostupné z: <https://mariadb.com/docs/analytics/mariadb-columnstore>. Online. MariaDB.
21. INFLUXDATA. *InfluxDB Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://docs.influxdata.com/influxdb/latest/>. Online. InfluxData.
22. IOTDB, Apache. *Apache IoTDB Documentation* [online]. [cit. 2025-11-10]. Dostupné z: https://iotdb.apache.org/UserGuide/latest/IoTDB-Introduction/IoTDB-Introduction_apache.html. Online. Apache IoTDB.
23. MARIADB. *MariaDB Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://mariadb.com/docs/>. Online. MariaDB.
24. MONGODB. *MongoDB Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://www.mongodb.com/docs/>. Online. MongoDB.
25. QUESTDB. *QuestDB Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://questdb.io/docs/>. Online. QuestDB.
26. TIMESCALEDDB. *TimescaleDB Documentation* [online]. [cit. 2025-11-10]. Dostupné z: <https://docs.timescale.com/>. Online. TimescaleDB.

A. Externí přílohy

Externí přílohy této bakalářské práce jsou umístěny na adrese:

https://github.com/Jiri-Fiser/thesis_ki_ujep.

Na úložišti GitHub mohou být uloženy tyto externí přílohy:

- **zdrojové kódy**
- **doplňkové texty** (například jak instalovat aplikaci, manuály aplikace)
- **schémata** (především, pokud se nevejdou na stranu A4 a jejich vytištění je tak problematické)
- **screenshoty** (v textu práce lze použít jen omezený počet snímků obrazovky, které navíc nemusí být při černobílém tisku příliš přehledné)
- **videa** (například ovládání aplikace)

V každém případě by to však měli být pouze materiály, které jste vytvořili sami. Materiály jiných autorů uvádějte v seznamu použité literatury (včetně případných odkazů na jejich originální umístění).

V této kapitole stačí uvést pouze základní strukturu úložiště (co se kde nalézá a jakou má funkci) například v podobě tabulky.

ki-thesis.pdf	text práce v PDF
ki-thesis.tex	zdrojový kód práce v $\text{\LaTeX}$
kitheses.cls	definice třídy dokumentů (rozšířená třída scrbook)
thesis.bib	bibliografická databáze (exportována z citace.com)
LOGO_PRF_CZ_RGB_standard.jpg	logo fakulty s českým textem
LOGO_PRF_EM_RGB_standard.jpg	logo fakulty s anglickým textem

Všechny tyto soubory jsou potřeba pro překlad dokumentu (logo stačí jedno v příslušné jazykové verzi).